

计算复杂性第三次编程作业规格说明

本次编程作业中，同学们需要以**小组**为单位，编写 Python 程序实现两个近似算法，并记录算法运行结果、运行时间和近似效果。

本次作业包含两个任务：

1. **任务 1：MAX-3SAT 的随机近似算法**，实现课件 Randomized Algorithm for Max3SAT-Williams.pdf 第 29 页中的 Las Vegas 风格随机算法：不断随机生成变量赋值，直到找到一组能够满足至少 $\frac{7}{8}$ 子句的变量赋值。
2. **任务 2：METRIC-TSP 的 2-近似算法**，实现课件 Approximation Algorithms.pdf 第 68 页到第 77 页中的基于最小生成树与 DFS 遍历的近似算法。

本次作业不规定同学们必须实现哪些模块、类或函数。各小组可以自行设计程序结构，但必须满足本文档规定的输入格式、输出规范、测试要求与提交要求。

任务 1：MAX-3SAT 的随机近似算法

MAX-3SAT 问题的输入是一个 3SAT 布尔公式，目标是找到一组变量赋值，使得被满足的子句数量尽可能多。

本任务中，同学们需要实现如下随机近似算法：

```
循环：
    随机生成一组变量赋值
    计算该赋值满足的子句数量
直到：
    满足的子句数量不少于全部子句数量的  $\frac{7}{8}$ 
输出：
    该组变量赋值、满足的子句数量、循环次数和运行时间
```

设输入公式共有 m 个子句，则程序应当持续随机生成变量赋值，直到找到一组满足至少 $\lceil \frac{7m}{8} \rceil$ 个子句的赋值。

MAX-3SAT 公式输入格式

程序应当支持从 txt 文件读取 MAX-3SAT 公式。一个 txt 文件代表一个测试用例。

统一采用如下输入格式：

```
x1 x2 x3
-x1 x2 x4
x1 -x3 -x4
-x2 x3 x4
```

格式说明如下：

1. 每一行表示一个子句。
2. 每一行必须包含恰好 3 个文字 (literal) ， 以空格分隔。
3. 正文字直接写变量名，例如 `x1`；负文字在变量名前加 `-`，例如 `-x1` 表示 $\neg x_1$ 。
4. 每个子句中的 3 个变量应当互不相同。
5. 输入公式至少包含 **4 个变量**和 **4 个子句**。

上面的输入文件对应公式：

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_1 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_3 \vee x_4).$$

MAX-3SAT 随机生成公式的输入文件格式

除了从 txt 文件读取已有公式外，程序还应当支持根据配置文件随机生成 MAX-3SAT 公式。

随机生成配置文件也采用 **txt 文件**，格式如下：

```
number_of_variables 8
number_of_clauses 20
seed 2026
output_file random_8_20.txt
```

格式说明如下：

1. `number_of_variables` 表示变量数量，必须为大于等于 4 的整数。
2. `number_of_clauses` 表示子句数量，必须为大于等于 4 的整数。
3. `seed` 表示随机种子，用于复现实验结果。
4. `output_file` 表示随机生成的公式保存路径。

随机生成的公式必须满足：

1. 每个子句恰好包含 3 个文字。
2. 同一个子句中不得出现重复变量。
3. 同一个子句中不得同时出现某个变量及其否定，例如不得出现 `x1 -x1 x2`。
4. 变量统一命名为 `x1, x2, ..., xn`。

5. 使用相同的 `seed`、`number_of_variables` 和 `number_of_clauses` 时，应当生成相同的公式。

MAX-3SAT 输出要求

每个测试用例运行后，程序至少需要输出以下信息：

输入公式：

$(x1 \vee x2 \vee x3) \wedge \dots$

变量数量：4

子句数量：4

目标满足子句数量：4

找到的变量赋值：

`x1 = True`

`x2 = False`

`x3 = True`

`x4 = True`

满足的子句数量：4 / 4

满足的子句编号：C1, C2, C3, C4

随机循环次数：17

运行时间：0.0031 秒

其中：

1. **随机循环次数**指程序一共生成了多少组随机赋值，直到第一次找到满足要求的赋值。
2. **运行时间**应当只统计近似算法核心运行过程的时间。

MAX-3SAT 测试要求

任务 1 至少需要包含以下测试用例：

1. **固定测试用例**：使用本文档给出的至少 4 个变量、4 个子句的 MAX-3SAT 公式。
2. **随机生成测试用例**：使用随机生成配置文件生成不同规模的 MAX-3SAT 公式，并使用程序完成求解，记录不同规模问题下的平均循环次数和运行时间。

建议同学们测试较大规模的随机公式，例如 8 个变量、20 个子句，以观察循环次数和运行时间随问题规模变化的情况。

任务 2：METRIC-TSP 的 2-近似算法

METRIC-TSP 问题的输入是一个满足三角不等式的带权完全图，目标是找到一条总权重尽可能小的回路，使得每个顶点都被访问至少一次。

本任务中，同学们需要实现课件 `Approximation Algorithms.pdf` 中基于最小生成树的 2-近似算法：

输入：

带权完全图 $G = (V, E)$ ，边权 c 满足三角不等式

算法：

1. 计算 G 的最小生成树 MST
2. 任选一个顶点 s 作为根
3. 从 s 开始对 MST 进行 DFS 遍历
4. 根据 DFS 遍历顺序构造 TSP 近似回路
5. 输出该回路及其总代价、最优解代价和近似比例

输出：

一条 METRIC-TSP 近似回路

程序需要输出最小生成树 MST、DFS 遍历顺序、TSP 近似回路、MST 代价、TSP 近似回路代价、最优 TSP 回路代价、近似比例和运行时间。

METRIC-TSP 输入格式

为便于生成满足三角不等式的实例，本次作业统一采用**二维坐标点**作为输入。程序根据点之间的欧氏距离自动构造带权完全图。

一个 txt 文件代表一个 METRIC-TSP 测试用例，示例如下：

```
a 0 0
b 2 3
c 5 1
d 6 5
e 8 2
f 1 6
```

格式说明如下：

1. 每一行表示一个城市，即一个顶点；
2. 每一行包含城市名称、横坐标、纵坐标三项；
3. 城市名称与坐标之间以空格分隔；
4. 输入实例至少包含 **6 个城市**；
5. 不同城市不应具有完全相同的坐标。

程序应当根据任意两个城市 (u, v) 的坐标计算边权：

$$c(u, v) = \left\lceil \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2} \right\rceil.$$

由此得到的完全图满足三角不等式，可以作为 METRIC-TSP 实例。

除上述示例外，同学们需要另外构造至少一个满足要求的测试用例用于测试。

METRIC-TSP 近似算法实现要求

程序应当完整实现以下步骤：

1. 构造带权完全图

根据输入城市坐标，计算任意两个城市之间的边权，构造带权完全图。

2. 计算最小生成树 MST

使用 Prim 算法或 Kruskal 算法计算 MST。同学们可以自行选择实现方式，但不能直接调用第三方库。

3. DFS 遍历 MST

从根节点开始对 MST 进行 DFS 遍历。

4. 构造 TSP 近似回路

根据 DFS 首次访问顶点的顺序构造回路，并在最后返回起点。例如：

```
DFS 遍历顺序：a -> b -> c -> d
TSP 回路：a -> b -> c -> d -> a
```

5. 计算 TSP 近似回路总代价

根据原始完全图中的边权计算该 TSP 回路的总代价。

6. 计算最优 TSP 回路总代价

为了给出近似比例，本次作业要求对测试用例计算最优 TSP 回路总代价。测试用例城市数量不宜过大，建议不超过 9 个城市。

7. 计算近似比例

输出算法得到的 TSP 近似回路代价与最优 TSP 回路代价之比：

$$\text{近似比例} = \frac{\text{TSP 近似回路总代价}}{\text{最优 TSP 回路总代价}}.$$

8. 记录运行时间

程序应当记录近似算法的总运行时间。

METRIC-TSP 输出要求

对于上一节给出的测试用例，若采用 Kruskal 算法计算 MST，一个参考输出如下：

城市数量：6

根节点：a

最小生成树 MST 边集：

(a, b, 4)

(b, c, 4)

(b, f, 4)

(c, e, 4)

(d, e, 4)

MST 总代价：20

DFS 遍历顺序：

a -> b -> c -> e -> d -> f

TSP 近似回路：

a -> b -> c -> e -> d -> f -> a

TSP 近似回路总代价：29

2 * MST 总代价：40

最优 TSP 回路：

a -> b -> f -> d -> e -> c -> a

最优 TSP 回路总代价：28

近似比例：29 / 28 = 1.036

运行时间：0.026s

实验报告要求

本次作业需要提交**代码**和**实验报告**。实验报告至少需要包含以下内容：

1. MAX-3SAT 近似算法思路；
2. 任务 1 的关键代码展示及说明；
3. 任务 1 的固定测试用例运行结果展示；
4. 任务 1 的随机生成测试用例运行结果展示；
5. 任务 1 中随机循环次数和运行时间的记录与分析；
6. METRIC-TSP 近似及精确算法思路；
7. 任务 2 的关键代码展示及说明；
8. 任务 2 的测试用例运行结果展示与分析；

项目目录

同学们的代码和实验报告应当组织成一个项目。项目目录建议组织如下：

```
.
├── src                                // 代码文件夹
├── test                              // 测试文件夹
│   ├── max3sat                      // MAX-3SAT 测试用例
│   │   ├── fixed.txt
│   │   └── random_config.txt
│   └── metric_tsp                  // METRIC-TSP 测试用例
│       └── sample.txt
├── output                          // 输出文件夹
│   ├── max3sat
│   └── metric_tsp
├── docs
│   └── report.pdf                  // 实验报告
├── requirements.txt                // 依赖库说明
└── README.md                      // 简单的项目说明文件，告诉助教如何运行程序
```

README.md 至少应当说明：

1. 如何安装依赖；
2. 如何运行 MAX-3SAT 测试；
3. 如何运行 METRIC-TSP 测试；

提交方式

实验报告中务必写上每一个小组成员的姓名和学号！！！！

电子版：每个小组将作业项目打包成一个压缩文件，命名格式为：

组长学号_组长姓名_计算复杂性第三次编程作业.zip

例如：

12345678_张三_计算复杂性第三次编程作业.zip

由组长发送至助教邮箱。

- 助教邮箱: linkh8@mail2.sysu.edu.cn

纸质版: 每个小组双面打印一份实验报告, **组长签字**, 于电子版截止日期后一天的课上提交。

截止时间

电子版作业截止时间为 **2026 年 6 月 23 日晚上 23:59**。请务必在此之前提交电子版作业项目。逾期将酌情扣分。

纸质版实验报告应当在 **2026 年 6 月 24 日课上**提交。逾期将酌情扣分。

评分标准

本次作业总分为 **30 分**。

任务 1: MAX-3SAT 随机近似算法 (15 分)

1. MAX-3SAT 输入解析与随机公式生成正确: 3 分
2. 随机变量赋值、子句满足判断和 $\frac{7}{8}$ 阈值判断正确: 5 分
3. 正确记录并输出循环次数、满足子句数量、变量赋值和运行时间: 1 分
4. 详尽的实验报告: 6 分

任务 2: METRIC-TSP 2-近似算法 (15 分)

1. 正确读取城市坐标并构造满足三角不等式的带权完全图: 1 分
2. MST 构造正确, 能够输出 MST 边集和总代价: 2 分
3. DFS 遍历实现无误、TSP 近似回路总代价计算正确: 2 分
4. 正确计算最优 TSP 回路总代价: 3 分
5. 正确记录运行时间和近似比例: 1 分
6. 详尽的实验报告: 6 分